

Projeto de Sistemas Operativos 2026-27

Enunciado da 1ª parte

LEIC-A/LEIC-T/LETI

O objetivo deste projeto é desenvolver o CloudIST, um sistema que simula o aluguer de máquinas virtuais num centro de dados.

Nos centros de dados modernos existe a possibilidade de alugar computadores para executar aplicações. O que é alugado não são máquinas físicas inteiras, mas sim parcelas de uma máquina física que simulam um computador completo, a que chamaremos *máquina virtual*. Uma máquina virtual (entre outras características) só pode usar uma fracção dos recursos da máquina física na qual se executa (processador, memória, disco, rede, etc). É importante notar que, no contexto deste projecto, não usaremos tecnologias de virtualização (virtualBox, Xen ou outro *hypervisor*) e aquilo que designamos por máquinas virtuais (VMs) serão simuladas simplesmente por processos (ou seja programas em execução) possivelmente com os seus recursos limitados como veremos adiante.

O CloudIST é um sistema que permite aos seus utilizadores reservar várias VMs simultaneamente num centro de dados e lançar a execução de aplicações nessas VMs. Os utilizadores terão a possibilidade de especificar quantas VMs pretendem reservar, quais as restrições de recursos (RAM, disco, e CPU) dessas VMs e que aplicações executarão. O CloudIST irá depois gerir as reservas das VMs, realizar tarefas preparatórias para o lançamento das aplicações e executar as aplicações de cada VM numa única máquina central que, no contexto deste trabalho, representará a totalidade do centro de dados.

O CloudIST explorará técnicas de paralelização baseadas em múltiplas tarefas (*threads*) de forma a acelerar a gestão dos pedidos de alocação das máquinas virtuais e recorrerá a processos para simular a execução das aplicações. Ao desenvolver o CloudIST, será necessário também aprender como implementar mecanismos de sincronização escaláveis entre tarefas bem como mecanismos de comunicação entre processos (filas FIFO e sinais). O CloudIST irá também interagir com o sistema de ficheiros oferecendo portanto a possibilidade de aprender a utilizar as interfaces de programação de sistemas de ficheiros POSIX.

Código base

O código base fornecido disponibiliza uma implementação incompleta de um programa que simula a gestão e execução do centro de dados de forma *sequencial*, com apenas uma tarefa.

A configuração inicial do centro de dados é realizada através dos argumentos da linha de comandos, passados aquando da execução do programa, utilizando a seguinte sintaxe:

```
cloudIST <SERVERS> <RAM> <DISK> <CPU>
```

onde:

- **SERVERS** - corresponde ao número de máquinas físicas do centro de dados;
- **RAM** - corresponde à memória RAM (em GB) disponível em cada máquina física;
- **DISK** - corresponde ao espaço em disco (em GB) disponível em cada máquina física;
- **CPU** - corresponde ao número de *CPU cores* (unidades de processamento) disponíveis em cada máquina física.

Assume-se que as máquinas físicas existentes num centro de dados são designadas sequencialmente (1, 2, ...) e que todas as máquinas físicas possuem os mesmos recursos.

Após a configuração inicial, a consola de simulação recebe um conjunto de comandos que descrevem as máquinas virtuais e iniciam a sua execução. Na versão do código base estes comandos são inseridos numa linha de comando. Os comandos disponíveis são:

- **Comando D** (definir): define um tipo de máquina virtual, especificando os recursos necessários (memória RAM, espaço em disco e número de vCPUs), o executável da aplicação a executar e a diretoria que contém os ficheiros necessários para a sua execução.

D <TIPO_VM> <DIR_INPUTS> <CAMINHO_EXEC> <RAM> <DISCO> <VCPU>

- **Comando R** (reservar): reserva um conjunto de máquinas virtuais pré-definidas com comando D. Caso não seja possível reservar todas as máquinas virtuais descritas num comando "R", nenhuma parte da reserva deverá ser realizada. Cada reserva terá uma cadeia de caracteres que a designa, e.g. "res_maria". Assume-se que as máquinas virtuais existentes numa reserva são designadas sequencialmente ("res_maria_1", "res_maria_2", ...).

pode se repetir as reservas (meter mais do que uma)

R <ID_RESERVA> [<TIPO_VM> <QTD> <ID_SERVIDOR>]+

se nao houver espaço para uma resreva especifica é necessario cancelar todas

- **Comando A** (arrancar): inicia a execução de uma reserva de máquinas virtuais previamente realizada.

A <ID_RESERVA>

- **Comando L** (listar): lista quais as máquinas em execução com toda a informação acerca delas.

L

- **Comando E** (esperar): espera até à execução do próximo comando durante o número de milissegundos especificados .

E <MS>

No primeiro exercício abaixo encontram-se mais detalhes sobre a sintaxe destes comandos.

1ª parte do projeto

A primeira parte do projeto consiste em 3 exercícios.

Exercício 1. Interação com o sistema de ficheiros

Neste exercício pretende-se estender o código base de forma a que passe a processar dados obtidos a partir de ficheiros.

Para este efeito, o CloudIST deverá passar a receber um argumento adicional na linha de comandos, correspondente ao caminho para a diretoria que contém os ficheiros de entrada. A sintaxe de execução passa a ser:

```
cloudIST <SERVERS> <RAM> <DISK> <CPU> <INPUT_DIR>
```

-Receber diretoria no argv
-Ver que ficheiros há na diretoria
-Ordenar por ordem alfabética
-Processar cada ficheiro e usar como input

onde:

- **INPUT_DIR** - corresponde à diretoria que contém os ficheiros de configuração;

O CloudIST deverá obter a lista de ficheiros com extensão `.conf` contidos na diretoria indicada na linha de comandos. Os ficheiros `.conf` contêm uma sequência de comandos para configurar e lançar máquinas virtuais. Estes ficheiros devem ser processados por ordem alfabética.

O acesso e a manipulação de ficheiros deverão ser efetuados através da interface POSIX baseada em descritores de ficheiros, e NÃO usando a biblioteca `stdio` (carregada através do ficheiro `stdio.h`) e a abstração de *FILE stream*. `execp` ainda esta por decidir

Sintaxe dos ficheiros de entrada

Todas as linhas iniciadas por `#` são comentários e devem ser ignoradas bem como todas as linhas vazias. Por simplicidade, assume-se que não há erros de sintaxe ou inconsistências nos ficheiros de entrada.

Exemplo de ficheiro de comandos, e.g. "1.conf":

```
# -----  
# COMANDO D (Definir Tipo de VM)  
# Sintaxe:  
# D <TIPO_VM> <DIR_INPUTS> <CAMINHO_EXEC> <RAM> <DISCO> <VCPUS>  
# -----  
# Exemplo 1: Define a VM "V1" (2GB RAM, 50GB Disco, 4 vCPUs)  
D V1 /tmp/v1-inputs /usr/bin/v1_app 2 50 4  
  
# Exemplo 2: Define a VM "Vx" (4GB RAM, 75GB Disco, 6 vCPUs)  
D Vx /tmp/vx-inputs /usr/bin/vx_app 4 75 6
```

```

# -----
# COMANDO R (Reservar VMs)
# Sintaxe:
# R <ID_RESERVA> [<TIPO_VM> <QTD> <ID_SERVIDOR>]+
# -----
# Exemplo: Reserva "R_inicial" contendo:
# - 4x VMs do tipo V1 no Servidor 1
# - 2x VMs do tipo Vx no Servidor 1
R R_inicial V1 4 1 Vx 2 1

# -----
# COMANDO A (Arrancar VMs)
# Sintaxe:
# A <ID_RESERVA>
# -----
# Exemplo: Executa as VMs de "R_inicial":
A R_inicial

# -----
# COMANDO L (Listar VMs)
# Sintaxe:
# L
# -----
# Exemplo: Lista o estado e detalhes de todas as VMs atualmente em execução
L

# -----
# COMANDO E (Esperar / Pausa)
# Sintaxe:
# E <MS>
# -----
# Exemplo: Aguarda 5000 ms (5 segundos) antes de processar o próximo comando
E 5000

```

Assume-se que no sistema de ficheiros da consola existem pastas associadas a cada VM. Essas pastas encontram-se no caminho:

```
/tmp/CloudIST/<ID-reserva>/<ID-VM>
```

Exemplo: /tmp/CloudIST/res_maria/res_maria_1

Antes da execução da aplicação, o CloudIST deverá copiar recursivamente todo o conteúdo da diretoria de entrada especificada no comando D para a diretoria correspondente à máquina virtual. Desta forma, quaisquer subdiretorias e respetivos ficheiros deverão ser preservados.

Para simplificar a implementação, assume-se que existirá apenas um comando **R** por cada ficheiro `.conf`.

Exercício 2. Execução concorrente

Após terem realizado o Exercício 1, os alunos devem estender o código criado de forma a que os programas de cada máquina se possam executar num processo separado.

fazer fork e exec com o ficheiro addo para correr em paralelo

Este procedimento deve ser implementado usando as chamadas de sistema `fork` e `exec` para que seja possível libertar o processo principal que está a percorrer os comandos dos ficheiros `.conf`.

Para limitar os recursos utilizados pelas máquinas virtuais, existem as seguintes ferramentas:

limita o numero de cpus da maquina que usa

- RAM e Disco (`setrlimit`): É recomendado o uso da chamada de sistema **`setrlimit`**, executada no processo filho antes de invocar o `exec`. Deverão consultar a página do manual (*man setrlimit*) para mais detalhes. Por exemplo, podemos limitar a RAM e o tamanho máximo por ficheiro para um processo da seguinte forma:

```
struct rlimit lim;

lim.rlim_cur = 512UL << 20; /* limite de 512MB de RAM alterável pelo processo */
lim.rlim_max = 512UL << 20; /* limite de 512 MB de RAM não alterável */
setrlimit(RLIMIT_AS, &lim);

lim.rlim_cur = 100UL << 10; /* limite de 100KB de tamanho máximo por ficheiro */
/* alterável pelo processo */
lim.rlim_max = 100UL << 10; /* limite de 100KB de tamanho máximo por ficheiro */
/* não alterável pelo processo */
setrlimit(RLIMIT_FSIZE, &lim);
```

- CPU (`cpulimit`): Para limitar a utilização de CPU, deverão utilizar o pacote `cpulimit`, que deverá ser instalado nos computadores pessoais (p.e.: `sudo apt install cpulimit`).
Sintaxe do comando `cpulimit`:

`cpulimit -q -f -l <CPULIMIT> -- <EXEC_PATH>`

Onde:

- **`-q`** - oculta as mensagens do estado do `cpulimit` no terminal
- **`-f`** - força a execução do `cpulimit` em primeiro plano
- **`-l <CPULIMIT>`** - Define a percentagem máxima de tempo de CPU que o processo pode usar
- **`<EXEC_PATH>`** - Especifica o caminho para o ficheiro executável da aplicação que será lançada com o limite de CPU

Assim, por exemplo, se se pretender executar uma aplicação, e.g. “apl1”, com o uso de CPU limitado (por exemplo a 10%), deve-se executar o comando `cpulimit` passando o nome da aplicação como parâmetro, ou seja: **`cpulimit -q -f -l 10 - apl1`**

Quando uma aplicação termina a sua execução, a VM respectiva deve ser imediatamente destruído pelo CloudIST. Isto implica que os recursos alocados à VM devem ser devolvidos

à máquina física correspondente, ficando disponíveis para novas reservas. Assim que todas as VMs de uma reserva terminarem, o registo da reserva deve ser totalmente apagado do sistema. Note-se que o comando `cpulimit` não controla o número de cores usados mas somente a percentagem de tempo de CPU da máquina física que se estará a usar e isso é suficiente no contexto do CloudIST. Assim, a percentagem de CPU a atribuir a cada VM para simular correctamente o número de CPU virtuais (vCPU) atribuídos no ficheiro de configuração deve ser: $vCPU-da-VM / n^o-de-CPUs-por-máquina-física \times n^o-de-máquinas-físicas-no-centro-de-dados \times 100$

Exercício 3. Paralelização usando múltiplas tarefas

O objetivo deste exercício é que a consola da simulação passe a executar vários ficheiros `.conf` simultaneamente.

O número máximo de ficheiros processados em paralelo deverá ser configurado através de um argumento da linha de comandos, que define o número de *worker threads* utilizadas pelo sistema. A sintaxe de execução passa a ser:

cloudIST <SERVERS> <RAM> <DISK> <CPU> <INPUT_DIR> <MAX_THREADS>

Onde:

- **MAX_THREADS** - corresponde ao número máximo de *worker threads* que poderão existir em simultâneo para o processamento dos ficheiros `.conf`;

Serão valorizadas soluções de sincronização no acesso ao estado do centro de dados que maximizem o grau de paralelismo atingível na reserva de máquinas virtuais.

Idealmente, este exercício deveria ser realizado a partir do código obtido após a resolução do exercício 2. Contudo, não serão aplicadas penalizações se a solução deste exercício for realizada a partir da solução do exercício 1.

Submissão e avaliação

A submissão é feita através do Fénix **até ao dia 23/10/2026 às 23h59**.

Os alunos devem submeter um ficheiro no formato zip com o código fonte e o ficheiro Makefile. O arquivo submetido não deve incluir outros ficheiros (tais como binários). Além disso, o comando *make clean* deve limpar todos os ficheiros resultantes da compilação do projeto.

Recomendamos que os alunos se assegurem que o projeto compila/corre corretamente no cluster *sigma*. Ao avaliar os projetos submetidos, em caso de dúvida sobre o funcionamento do código submetido, os docentes usarão o cluster *sigma* para fazer a validação final.

O uso de outros ambientes para o desenvolvimento/teste do projeto (e.g., macOS, Windows/WSL) é permitido, mas o corpo docente não dará apoio técnico a dúvidas relacionadas especificamente com esses ambientes.

A avaliação será feita de acordo com o método de avaliação descrito no *site* da cadeira.

Os alunos não podem partilhar código e ou soluções com outros grupos. O código submetido tem de ser o resultado do trabalho original de cada grupo. A submissão de código com grande grau de semelhança com outros grupos ou realizado recorrendo a entidades externas ao grupo levará à reprovação dos grupos envolvidos e ao reporte da situação à coordenação da LEIC e ao Conselho Pedagógico do IST.